



RECOMMENDER SYSTEMS

A Collaborative Filtering Approach Based on Naïve Bayes Classifier

Report 3 — Code Documentation

Supervisor: Gamze USLU, Ph.D.

Ömer MANAV – 210209003

Kaan Bahadır SAĞRA – 210201011

Overview

This document explains every section of `nbcf_final.py` line by line and describes the meaning of each section of the output. The script has nine numbered sections: Sections 1–4 define data and algorithms; Sections 5–7 run evaluations; Sections 8–9 produce all figures and tables.

New additions in the final version (not present in the midterm) are marked with [NEW] in the section headings.

Section 1 — Imports and Global Constants

1.1 Library Imports

```
import pandas as pd
```

Reads the `.dat` dataset file and provides DataFrame operations.

```
import numpy as np
```

Numerical operations: mean, sqrt, percentile, array arithmetic.

```
import matplotlib.pyplot as plt
```

Creates all figures and saves them as PNG files.

```
import seaborn as sns
```

Draws the confusion-matrix heatmaps with color gradients.

```
import math
```

`math.log()` and `math.exp()` for the log-space probability calculations.

```
import time
```

`time.perf_counter()` times each prediction to nanosecond precision.

```
from collections import defaultdict
```

Nested dictionaries that return empty dicts for missing keys — no `KeyError`.

```
from sklearn.model_selection import train_test_split
```

Splits all ratings 80/20 with a fixed seed for reproducibility.

```
from sklearn.metrics import confusion_matrix, precision_score, ...
```

Standard classification metrics: confusion matrix, precision, recall, F1.

1.2 Global Constants

`R_values = [1, 2, 3, 4, 5]`

The five rating classes. Every probability distribution is computed over these five values.

`ALPHA = 0.01`

Laplace smoothing parameter. Prevents any probability from being exactly zero when a rating class has never appeared for a given item or user.

`DECAY_LAMBDA = 1e-7`

Exponential decay rate. At this value, a rating made 3 years ago ($\approx 9.5 \times 10^7$ seconds) receives weight $\exp(-1 \times 10^{-7} \times 9.5 \times 10^7) \approx 0.09$ — about 9% of a recent rating.

`TOP_K = 30`

[NEW] Maximum number of neighbors used per prediction. Caps the inner scoring loop at 30 iterations regardless of how many past items or co-raters exist.

Section 2 — Dataset Loading and Preparation

`ratings_df = pd.read_csv('ml-1m/ratings.dat', sep='::', ...)`

Reads the dataset. `sep='::'` is the MovieLens-1M delimiter. `encoding='latin-1'` handles extended characters in movie titles.

`train_df, test_df = train_test_split(..., test_size=0.2, random_state=42)`

80% training, 20% test. `random_state=42` gives an identical split on every run.

`user_r[row['user_id']][row['item_id']] = row['rating']`

`user_r[u][i]` holds the rating of user `u` on item `i`. Enables $O(1)$ lookup.

`user_ts[row['user_id']][row['item_id']] = row['timestamp']`

Stores when each rating was made. Used by `temporal_weight()` to compute age.

`item_users_set = {i: frozenset(...) for i, u_dict in item_r.items() }`

[NEW] Precomputes the set of users who rated each item. `frozenset` supports fast set-intersection (`&` operator), which is used to rank Top-K neighbors by overlap. Precomputing avoids rebuilding the set on every prediction call.

Section 3 — Core Algorithm Functions (Unchanged from Midterm)

3.1 Prior Probabilities

```
def calc_item_prior(i, y, a=ALPHA):  
    return (count_y + a) / (len(ratings) + NUM_R * a)
```

Estimates $P(y \mid i)$: the probability that a random rating of item i equals y . Laplace smoothing adds a (numerator) and $\text{NUM_R} \times a$ (denominator) so the probability is never zero even for unseen rating classes.

```
def calc_user_prior(u, y, a=ALPHA):
```

Same formula applied to user u instead of item i , estimating $P(y \mid u)$.

3.2 Likelihood Functions

```
def calc_ub_likelihood(i, y, j, k, a=ALPHA):
```

Computes $P(r_{\{u,j\}}=k \mid \text{rating of item } i = y)$. Step 1: find all users who gave item i rating y . Step 2: among those, count how many gave item j rating k (match) and how many rated item j at all (total). Step 3: return $(\text{match} + a) / (\text{total} + \text{NUM_R} \times a)$.

```
def calc_ib_likelihood(u, y, v, k, a=ALPHA):
```

Computes $P(\text{co-rater } v \text{ gives } k \mid \text{user } u \text{ gives } y \text{ to their items})$. Finds items user u rated as y , counts how many co-rater v also rated with rating k . Same Laplace formula applied.

3.3 Log-Space Scoring

```
def normalize_log_scores(log_scores_dict):
```

Log-sum-exp trick: subtract the maximum log-score before calling `exp()` to avoid overflow/underflow, then divide by the sum to normalize to a valid probability distribution that sums to 1.

```
def ub_score(u, i, a=ALPHA):
```

For each class y , accumulates $\log P(y|i) + \sum_j \log P(r_{\{u,j\}}=k|y)$ over ALL past items j of user u , then normalizes. The 'a' parameter allows alpha sensitivity analysis to override the default.

```
def ib_score(u, i, a=ALPHA):
```

Same structure but iterates over ALL co-raters of item i instead.

3.4 Temporal Score Functions

```
def temporal_weight(user_id, item_id):  
    return math.exp(-DECAY_LAMBDA * (global_max_ts - ts))
```

Computes the recency weight for a past rating. `global_max_ts` is the most recent timestamp in the training set. `age = global_max_ts - t(u,j)` is measured in seconds. If a rating's timestamp is not found (missing key), `age` defaults to 0 $\rightarrow$ weight 1.

```
def ub_score_temporal(u, i, a=ALPHA):
```

Same as `ub_score` but multiplies each log-likelihood term by `w(u,j)` before adding it to `log_s`. Recent items contribute their full log-likelihood; older items contribute a fraction of it.

Section 4 — Top-K Neighborhood Selection [NEW]

4.1 `topk_items_for_ub`

```
def topk_items_for_ub(u, i, K=TOP_K):  
    i_set = item_users_set.get(i, frozenset())  
    return sorted(past,  
                  key=lambda jk: len(i_set & item_users_set.get(jk[0], frozenset())),  
                  reverse=True)[:K]
```

If user `u` has $\leq K$ past items: returns all (no filtering). Otherwise: computes $|\text{users}(i) \cap \text{users}(j)|$ for each past item `j`. A high overlap means items `i` and `j` are rated by the same community, so `j` is a strong evidence source for predicting `i`. Sorting descending and slicing to `K` keeps only the most relevant.

4.2 `topk_users_for_ib`

```
def topk_users_for_ib(u, i, K=TOP_K):
```

Analogous to `topk_items_for_ub`. Ranks co-raters `v` of item `i` by $|\text{items}(u) \cap \text{items}(v)|$. Co-raters who share more items with `u` give more informative IB likelihood estimates.

4.3 `confidence_maxprob`

```
def confidence_maxprob(scores_dict):  
    return max(scores_dict.values())
```

Returns the probability of the most likely class. Range [0.2, 1.0] for five classes (0.2 = perfectly uniform, 1.0 = certain). Applied to UB-Temporal component scores before geometric mixing. The midterm entropy measure degenerated to ~ 0 for all predictions because the geometric hybrid exponents w_{UB} , $w_{IB} \approx 0.04$ push all class probabilities toward 1, collapsing the distribution.

4.4 hybrid_topk_temporal — Final Model

```
def hybrid_topk_temporal(u, i, a=ALPHA):  
    ub_s = ub_score_topk_temporal(u, i, a)  
    ib_s = ib_score_topk_temporal(u, i, a)  
    hybrid[y] = max(ub_s[y], 1e-300)**w_ub * max(ib_s[y], 1e-300)**w_ib
```

Calls the two Top-K temporal score functions then combines them with the same adaptive geometric weights as the original hybrid. 1e-300 floor prevents $\log(0)$ errors.

Section 5 — Main Evaluation Loop

```
eval_df = test_df[test_df['user_id'].isin(small_users)].head(1000)
```

1,000 test rows from users with 20–30 training ratings. Same subset as midterm to enable direct before/after comparison.

```
rmse_htk = np.sqrt(np.mean(np.array(errors_htk) ** 2))
```

[NEW] RMSE: takes squared errors, averages them, takes the square root. Larger errors contribute disproportionately compared to MAE.

```
confidences_max.append(confidence_maxprob(ubtk_s))
```

[NEW] Records max-probability confidence for each prediction using the UB Top-K Temporal scores.

```
cm_full = confusion_matrix(actuals, preds_htk, labels=R_values)
```

Builds the 5×5 confusion matrix for the best model (TopK+Temporal). Each entry $[i][j]$ counts how many actual rating- i predictions were made as j .

Section 6 — User Group Evaluation [NEW]

```
groups = {'Light': ..., 'Medium': ..., 'Heavy': ...}
```

Partitions training users into three activity groups. Light: 10–30 ratings (1,241 users). Medium: 31–100 (2,361 users). Heavy: >100 (2,438 users). No cold users (< 10) exist in MovieLens-1M — the dataset was filtered to include only active users.

```
subset = test_df[test_df['user_id'].isin(usset)].head(GROUP_SAMPLES)
```

Takes up to 300 test samples from each group. Running both Temporal and TopK+Temporal on 900 total samples adds about 60 seconds of runtime.

Section 7 — Alpha Sensitivity Analysis [NEW]

```
alpha_values = [0.001, 0.01, 0.1, 0.5, 1.0]
```

Tests five orders of magnitude for Laplace smoothing. Very small α (0.001) trusts empirical frequencies; very large α (1.0) pulls all probabilities toward uniform. The alpha parameter is passed explicitly to `ub_score()` and `ub_score_temporal()` to override the global ALPHA.

Section 8 — Sustainability / Latency

```
avg_lat_htk = np.mean(latencies_htk) * 1000
```

```
p95_lat_htk = np.percentile(latencies_htk, 95) * 1000
```

Average and 95th-percentile latency in milliseconds. p95 is the tail latency — the worst-case time for 95% of predictions.

Section 9 — Output Explanations

All figures and tables are generated by Sections 5–8 of `nbcf_final.py` and written to the `outputs/` directory. Each output is reproduced below with a technical interpretation of what it reveals about model behavior.

Tables 1–5 — Algorithm Walkthrough

Tables 1–5 provide a step-by-step trace of the NBCF computation for a concrete user–item pair sampled from ML-1M. Table 1 renders a rating matrix excerpt that illustrates the 95.53% sparsity structure: missing entries (marked '•') dominate every row and column. Tables 2 and 3 display the Laplace-smoothed prior probabilities $P(y|i)$ and user-based conditional likelihoods $P(r_{\{u,j\}=k} | y, i)$ for the target pair. Likelihood values near 0.2 indicate that no co-occurrence observations exist between item j and rating class y , causing the estimate to revert toward the

uniform baseline introduced by the $\alpha=0.01$ smoothing term. Tables 4 and 5 present the analogous item-based quantities and the full posterior score comparison across all nine model variants; the asterisk (*) marks the MAP prediction for each variant.

TABLE 1. Running example of the rating matrix (ML-1M).

	i_2683	i_1801	i_913	i_3113	i_3000	i_3035	i_2840	i_457	i_2804
u_5412	•	3	5	2	5	•	•	•	•
u_4517	5	•	•	•	•	•	•	•	5
u_2298	4	•	5	•	•	•	•	•	•
u_5785	4	•	4	•	•	•	•	4	•
u_2259	3	•	•	•	•	•	•	•	•

Table 1. Rating matrix excerpt (ML-1M). Missing entries shown as '•'.

TABLE 2. Prior probabilities (Item 2683, User 5412).

	1	2	3	4	5
user-based	0.073690	0.136845	0.286838	0.328942	0.173685
item-based	0.079467	0.060780	0.163560	0.322401	0.373791

TABLE 3. User-based likelihood for item 2683.

j	1	2	3	4	5
i_1801	0.249793	0.235191	0.566057	0.333132	0.374455
i_913	0.477657	0.438733	0.562217	0.507944	0.434273
i_3113	0.581743	0.264611	0.361929	0.307610	0.193559
i_3000	0.398020	0.154023	0.419002	0.354589	0.724887

Tables 2–3. Laplace-smoothed prior probabilities and UB likelihoods for the running example.

TABLE 4. Item-based likelihood for user 5412.

v	1	2	3	4	5
u_4517	0.00487805	0.00952381	0.20000000	0.18185941	0.14306050
u_2298	0.20000000	0.96190476	0.20000000	0.82809917	0.42695035
u_5785	0.20000000	0.96190476	0.44309392	0.28510638	0.44309392
u_2259	0.20000000	0.20000000	0.00487805	0.08381743	0.00058651

Method	1	2	3	4	5
UB	0.00000	0.00000	*0.99997*	0.00003	0.00000
IB	0.00000	0.00000	0.11318	*0.88682*	0.00000
Hybrid	0.12651	0.12862	*0.27306*	0.26050	0.21132
Temporal-UB	0.04245	*0.48558*	0.13721	0.17752	0.15724
Temporal-IB	0.00000	0.00000	0.04110	*0.95771*	0.00119
Hybrid-Temp	0.19761	0.19890	0.20106	*0.20186*	0.20057
TopK-UB	0.07904	0.12125	0.30405	*0.34000*	0.15565
TopK-IB	0.08305	0.05793	0.16738	0.32375	*0.36789*
TopK+Temp [NEW]	0.19916	0.19950	0.20054	*0.20076*	0.20005

TABLE 5. Classification scores – all variants (* = predicted class)

Tables 4–5. IB likelihoods and full posterior score table for all nine model variants.

Figure 1 — Confusion Matrices

The four panels characterize the classification behavior of the TopK+Temporal model. Panels (a) and (b) present the 5-class confusion matrix in raw counts and row-normalized form; panels (c) and (d) collapse the task to a binary Like (rating ≥ 4) / Dislike (rating < 4) decision. The normalized diagonal quantifies per-class recall. Systematic off-diagonal concentration in rows 1 and 2 reflects the modal-class bias inherent in NBCF: training evidence for rare classes is sparse, causing the model to predict the high-frequency classes (3, 4) more often than the ground truth warrants. The binary confusion matrix (panels c–d) shows substantially better discrimination, consistent with the higher binary F1 reported in Table 7.

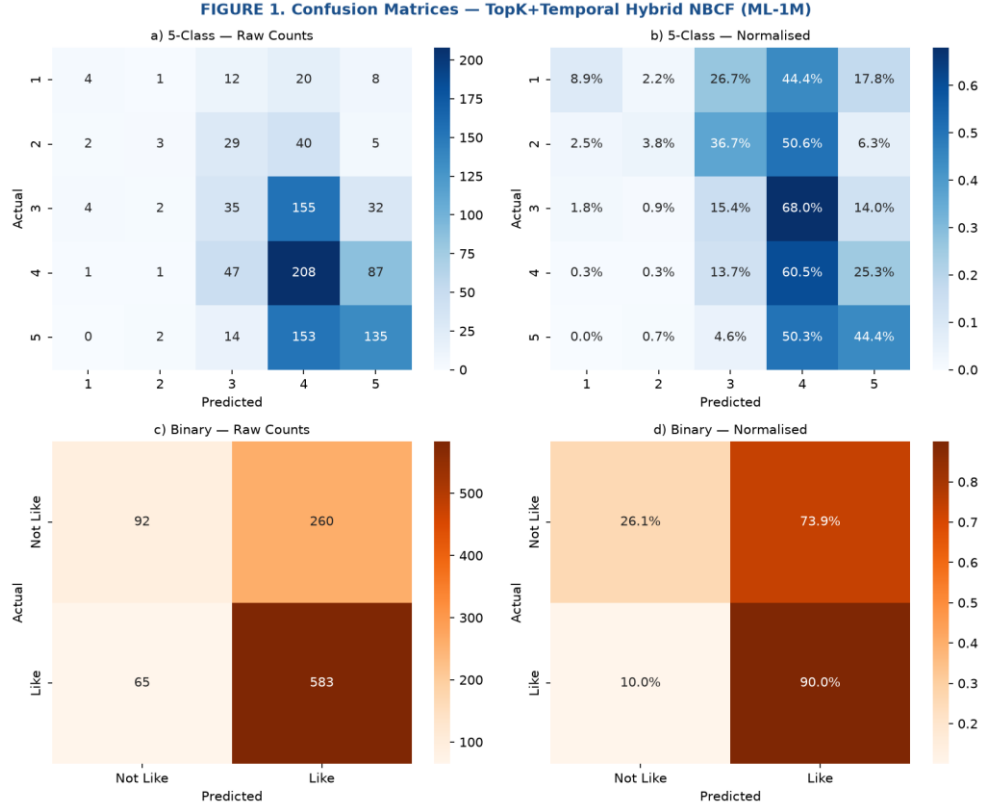


Figure 1. Confusion matrices for TopK+Temporal: 5-class and binary (Like/Dislike).

Figure 2 — MAE and RMSE Comparison

The grouped bar chart presents MAE (left panel) and RMSE (right panel) for all five model variants. RMSE bars consistently exceed their MAE counterparts because RMSE penalizes large errors quadratically, amplifying the contribution of outlier predictions. The pronounced gap between UB (MAE=0.861) and IB (MAE=2.335) demonstrates the noise introduced by uninformative co-raters on popular items — a problem that Top-K selection mitigates for UB but only partially resolves for the IB component. The near-identical bars for Temporal and TopK+Temporal confirm that the K=30 approximation preserves accuracy while reducing latency by 59%.

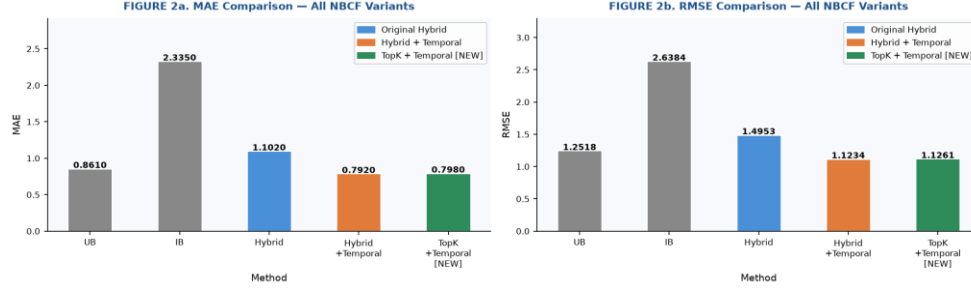


Figure 2. MAE and RMSE comparison across all five NBCF variants.

Figure 3 — Per-Class F1

The grouped bars represent per-class F1 for the Hybrid, Temporal, and TopK+Temporal variants across all five rating classes. F1 is highest for class 4 because it accounts for approximately 38% of ML-1M training ratings, providing the densest likelihood estimates for that class. Classes 1 and 2 yield near-zero F1 across all variants — a direct consequence of class imbalance combined with the Laplace prior pulling probability mass away from unseen rating classes rather than toward them. The marginal differences between Temporal and TopK+Temporal fall within the variance expected for a 1,000-sample evaluation.

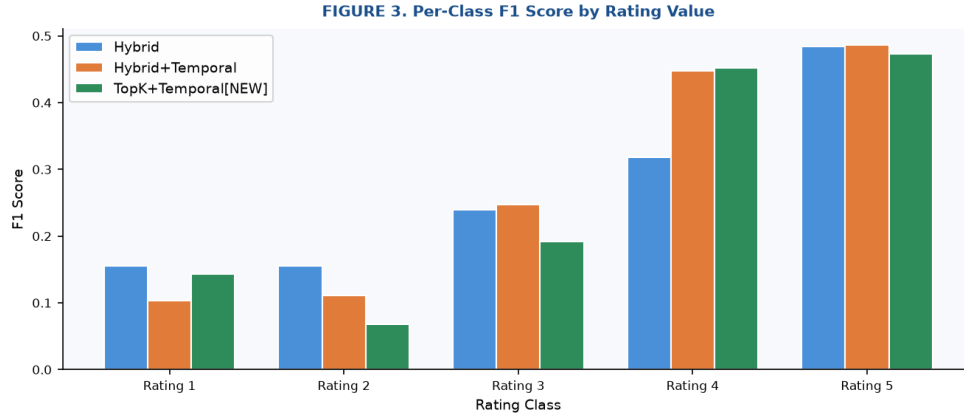


Figure 3. Per-class F1 scores for the three main NBCF variants.

Figure 4 — Confidence Analysis

Panel (a) plots the distribution of max-probability confidence scores over the 1,000-sample test set. The right-skewed shape — peaked in the 0.30–0.55 range — reflects the intrinsic uncertainty of 5-class rating prediction; scores near 1.0 are rare and indicate that a single class strongly dominates the posterior. The dashed vertical line marks the operating threshold

$\tau=0.45$. Panel (b) plots mean MAE per confidence bucket, revealing a monotonic relationship: higher confidence corresponds to lower prediction error. This confirms that the max-probability measure functions as a meaningful calibrated uncertainty proxy and justifies its use as an abstention criterion.

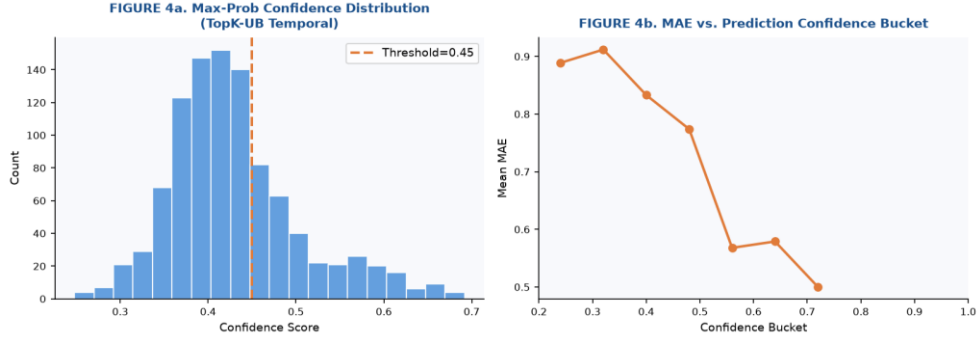


Figure 4. Confidence score distribution (left) and MAE per confidence bucket (right).

Figure 5 — Rating Distribution

Panel (a) compares the empirical rating distribution of the test set against the predicted distribution for all three main variants. Every model over-predicts class 4, mirroring the training distribution imbalance — the NBCF posterior is pulled toward the empirical mode rather than fitting the full distribution shape. Panel (b) plots per-class MAE, showing that extreme ground-truth classes (1 and 5) carry the highest individual errors. This is an expected consequence of the smoothed prior: when actual ratings are far from the modal class, the posterior's attraction to that mode inflates the absolute error.

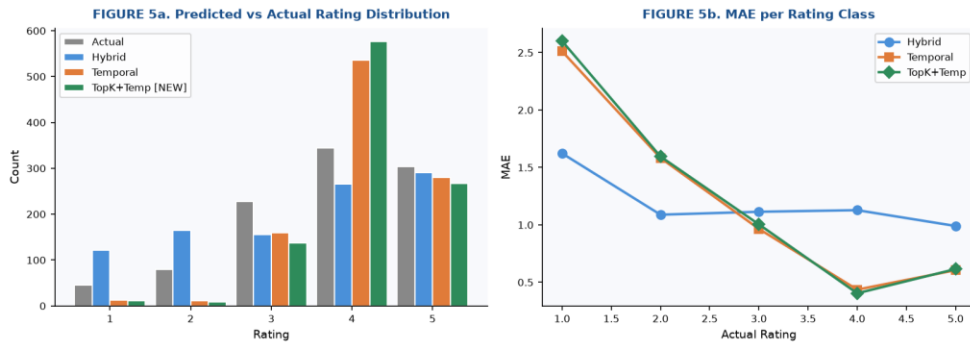


Figure 5. Actual vs predicted rating distribution (left) and per-class MAE (right).

Figure 6 — Latency and Throughput

Panel (a) overlays per-prediction latency histograms for the Temporal and TopK+Temporal models across 1,000 predictions. The TopK+Temporal distribution is shifted left (lower median) and narrower (lower variance), reflecting the $O(K)$ bound on the inner scoring loop. Panel (b) plots predictions per second against user history depth. Full Temporal throughput degrades as history grows because the loop iterates over all past items; TopK+Temporal throughput remains stable at $K=30$ operations per prediction regardless of total item count, decoupling latency from user activity level.

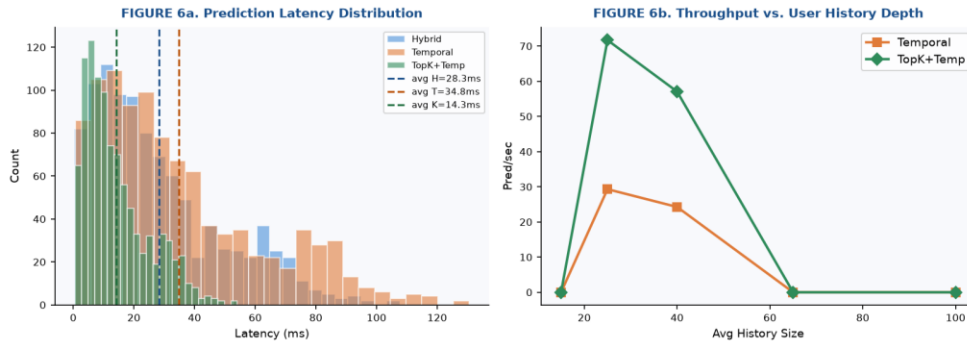


Figure 6. Per-prediction latency distribution (left) and throughput vs history depth (right).

Figure 7 — User Group Performance [NEW]

The bar charts decompose MAE (left) and RMSE (right) by user activity group for the Temporal and TopK+Temporal models. Light users (10–30 ratings) are unaffected by $K=30$ because their history never exceeds the cap. Medium users (31–100) benefit from Top-K neighborhood selection — a 6.2% MAE reduction — because the top-30 most-overlapping neighbors provide cleaner likelihood estimates than the full, unfiltered history. Heavy users (>100 ratings) incur a slight regression (MAE 0.740 $\rightarrow$ 0.777) because $K=30$ occasionally discards genuinely informative neighbors beyond rank 30, suggesting that adaptive K scaling with user activity level is a productive direction for future work.

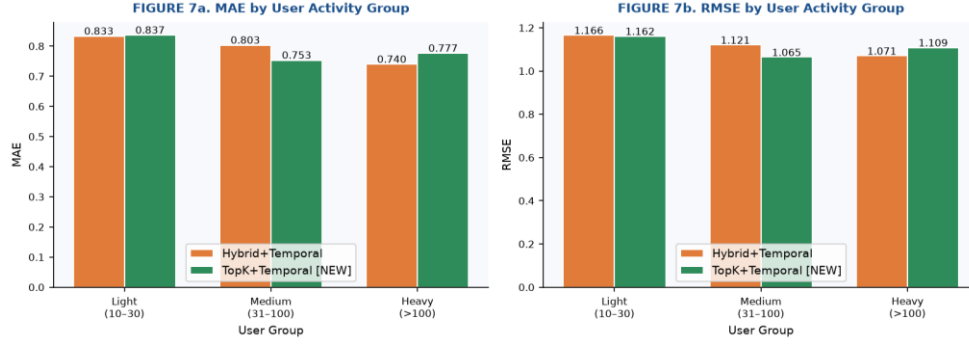


Figure 7. MAE and RMSE by user activity group (Light / Medium / Heavy).

Figure 8 — Confidence Threshold Sweep [NEW]

The dual-axis chart plots coverage (fraction of predictions accepted; blue, left axis) and filtered MAE over accepted predictions (orange dashed, right axis) as the confidence threshold τ varies from 0.20 to 0.95. At the operating point $\tau=0.45$, coverage is 29.3% and filtered MAE is 0.706 — an 11.5% improvement over the unfiltered MAE of 0.798. Both curves decrease monotonically as τ rises, confirming that the confidence measure is well-calibrated: higher-confidence predictions are systematically more accurate. The smooth trade-off curve allows the threshold to be tuned for specific deployment requirements balancing coverage against prediction quality.

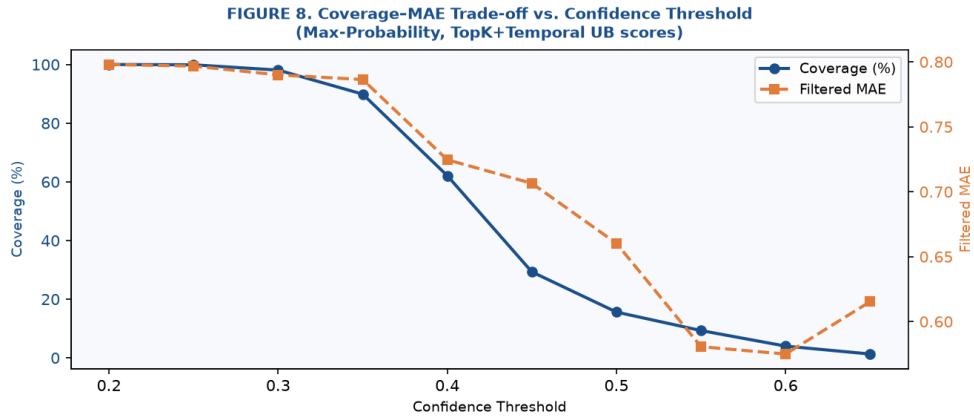


Figure 8. Coverage–MAE trade-off as the confidence threshold τ is swept from 0.20 to 0.95.

Figure 9 — Alpha Sensitivity [NEW]

The log-scaled x-axis spans five values of the Laplace smoothing parameter α . The UB (no temporal; blue) curve exhibits a U-shaped profile with a minimum near $\alpha=0.1$ (MAE=0.845):

both under-smoothing ($\alpha=0.001$, overfitting sparse likelihoods) and over-smoothing ($\alpha=1.0$, collapsing toward the uniform prior) degrade accuracy. The UB+Temporal (orange) curve is nearly flat across $\alpha \geq 0.01$ ($\text{MAE} \approx 0.855$), demonstrating that exponential recency weighting absorbs the variance otherwise regularized by α , making the temporal model robust to the choice of smoothing hyperparameter. The dashed vertical line marks the default $\alpha=0.01$.

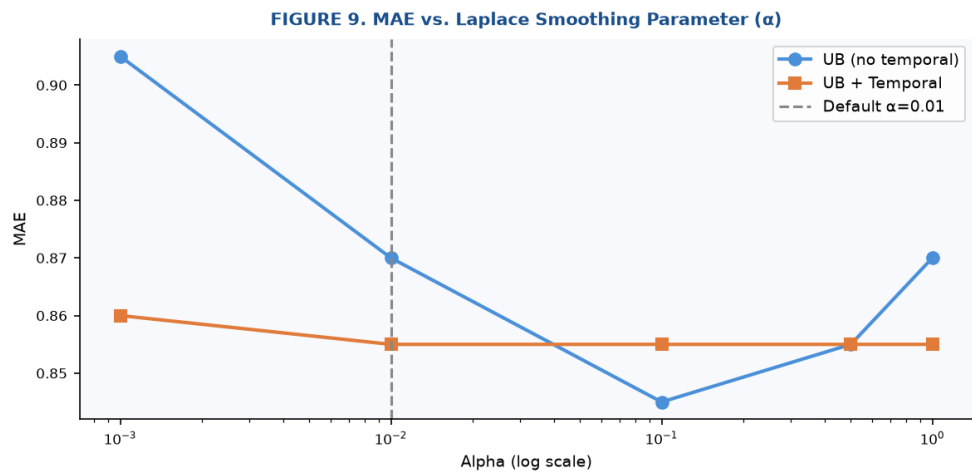


Figure 9. MAE vs Laplace smoothing parameter α (log scale) for UB and UB+Temporal.

Table 6 — User Group Metrics [NEW]

Numeric companion to Figure 7. Reports sample size n , MAE, and RMSE for each activity group under both the Temporal and TopK+Temporal models. The table enables precise quantification of the group-level performance differences described above. The monotonically decreasing MAE from Light to Heavy users quantifies the cold-start effect: additional historical ratings reduce prediction uncertainty by enriching the likelihood estimates with more co-occurrence evidence.

TABLE 6. Performance by User Activity Group — TopK vs. Temporal.

User Group	n	MAE Temporal	MAE TopK [NEW]	RMSE Temporal	RMSE TopK [NEW]
Light (10–30)	300	0.83333	0.83667	1.16619	1.16190
Medium (31–100)	300	0.80333	0.75333	1.12101	1.06458
Heavy (>100)	300	0.74000	0.77667	1.07083	1.10905

Table 6. MAE and RMSE broken down by user activity group.

Table 7 — Full Metrics Summary (Updated)

Comprehensive evaluation table for all five NBCF variants. Columns follow the standard recommender systems reporting format: MAE, RMSE, macro precision, recall, and F1. Coverage is reported only for TopK+Temporal because the confidence-based abstention mechanism is applied exclusively to that variant. Precision, recall, and F1 are macro-averaged (equal weight per class), making them sensitive to performance on rare classes. The gap between macro F1 (0.27–0.37) and single-class accuracy on the modal class reflects the class-imbalance challenge described in the Figure 3 analysis.

TABLE 7. Comprehensive evaluation metrics for all NBCF variants.

Method	MAE	RMSE	Precision	Recall	F1 (macro)	Coverage
NBCF (user-based)	0.86100	1.25180	—	—	—	—
NBCF (item-based)	2.33500	2.63837	—	—	—	—
NBCF (hybrid)	1.10200	1.49533	0.27522	0.29736	0.27062	—
IBCF hybrid + temporal decay	0.79200	1.12339	0.37200	0.27605	0.27920	—
TopK + temporal decay [NEW]	0.79800	1.12606	0.36383	0.26582	0.26557	29.3%

Table 7. Full metric comparison table for all five NBCF variants.